SymPy: Matemáticas Simbólicas

Aplicación Práctica: Facturación en Colombia

Mauricio Javier Contreras Ayala
Código: **20251020144**

Universidad Distrital Francisco José de Caldas

2 de marzo de 2026

¿Qué es SymPy?

- Es una biblioteca de Python para álgebra computacional.
- Mantiene la precisión exacta (simbólica) en lugar de aproximaciones decimales.
- Es libre, gratuita y multiplataforma.

Simbólico vs Numérico

En Python normal:

```
import math
print(math.sqrt(8)) # 2.828427...
```

En SymPy:

```
import sympy
print(sympy.sqrt(8)) # 2*sqrt(2)
```

Ejemplo Cotidiano: Facturación en Colombia

SymPy permite simplificar reglas tributarias (IVA 19 % y Retención 10 %):

```
from sympy import symbols, simplify

# Definimos el valor bruto (antes de impuestos)
Vb = symbols('Vb')

# Formula: Neto = Bruto + IVA - Retencion
neto = Vb + (0.19 * Vb) - (0.10 * Vb)

# SymPy simplifica la formula para cualquier cobro:
print(simplify(neto))  # Resultado: 1.09 * Vb
```

Conclusión del Modelo

Gracias a la simplificación simbólica, sabemos que el pago neto siempre será el 109 % del valor base.

Definiendo Símbolos

```
from sympy import symbols
x, y = symbols('x y')
expr = x + 2*y
```

Esto permite que Python trate a 'x' e 'y' como entes matemáticos, no como números.

Expandir y Factorizar

```
from sympy import expand, factor
expr = (x + 1)**2
# Expandir:
expand(expr) # x**2 + 2*x + 1
# Factorizar:
factor(x**2 - 1) # (x - 1)*(x + 1)
```

Simplificación

SymPy busca la forma más corta de una expresión:

```
from sympy import simplify, sin, cos
simplify(sin(x)**2 + cos(x)**2) # 1
```

Cálculo: Derivadas

Se usa la función `diff(función, variable)`:

```
from sympy import diff, sin
diff(sin(x), x) # cos(x)
```

Cálculo: Integrales

Indefinidas y definidas:

```
from sympy import integrate, exp
# Indefinida
integrate(exp(x), x) # exp(x)
# Definida de 0 a 1
integrate(x**2, (x, 0, 1)) # 1/3
```

Límites

```
from sympy import limit, oo
# Limite cuando x tiende a infinito
limit(1/x, x, oo) # 0
```

Resolución de Ecuaciones

Se usa `solve()` para hallar las raíces:

```
from sympy import solve
# Resolver x^2 - 4 = 0
solve(x**2 - 4, x) # [-2, 2]
```

Álgebra Lineal

```
from sympy import Matrix
A = Matrix([[1, 2], [3, 4]])
A.det() # Determinante: -2
A.inv() # Inversa
```

Ecuaciones Diferenciales

```
from sympy import Function, dsolve
f = Function('f')
# f''(x) + f(x) = 0
dsolve(f(x).diff(x, 2) + f(x), f(x))
```

Conversión a NumPy

Para graficar o hacer cálculos rápidos:

```
from sympy import lambdify
f_num = lambdify(x, sin(x), "numpy")
```

Series de Potencias

```
from sympy import exp
exp(x).series(x, 0, 3)
# 1 + x + x**2/2 + O(x**3)
```

Gráficos Integrados

```
from sympy.plotting import plot
plot(x**2, (x, -2, 2))
```

Ventajas

1. Integración total con Python.
2. No requiere licencias costosas.
3. Excelente para docencia e investigación.

Desventajas

- Más lento que el cálculo numérico puro.
- Curva de aprendizaje inicial para definir símbolos adecuadamente.

Ecosistema Python

SymPy es la base de otras librerías:

- **SciPy**: Para optimización.
- **PyDy**: Para dinámica multicuerpo.
- **Galgebra**: Álgebra geométrica.

Conclusión

SymPy cierra la brecha entre el papel y el código, permitiendo que la computadora entienda las matemáticas de forma abstracta.

¿Preguntas?

Gracias por su atención.

Mauricio Javier Contreras Ayala
Código: **20251020144**

Accede al código y documentación en GitHub:
https://github.com/mauriciocontreras083-cloud/Tarea-de-biblioteca.gi